



DDC606 - ANÁLISE DE ALGORITMO

ANÁLISE DE DESEMPENHO E COMPLEXIDADE DE ALGORITMOS PARA O CÁLCULO DA **N-ÉZIMO TERMO DE FIBONACCI**

Andersson Pereira, Igor Santos

Boa Vista - RR

2026

A SEQUÊNCIA DE FIBONACCI

- **Origem Histórica:** Introduzida no ocidente por Leonardo de Pisa (séc. XIII), com registros anteriores na matemática indiana (KNUTH, 1997).
- **Definição:** Cada termo é a soma algébrica dos dois imediatamente anteriores (CORMEN et al., 2009).
- **Aplicações:** Forte correlação com a proporção áurea, usada como modelo em análise de complexidade e estruturas de dados.

$$F(n) = \begin{cases} 0, & \text{se } n = 1 \\ 1, & \text{se } n = 2 \\ F(n - 1) + F(n - 2), & \text{se } n > 2 \end{cases}$$

IMPLEMENTAÇÃO RECURSIVA BÁSICA

```
função fibonacci(n):  
    se n <= 1:  
        retorne n  
    senão:  
        retorne fibonacci(n - 1) + fibonacci(n - 2)
```

Fonte: Elaborado pelos autores (2026).

FUNÇÃO DE CUSTO E COMPLEXIDADE

Função de custo:

$$T(n) = \begin{cases} c_1, & \text{se } n \leq 2 \\ T(n-1) + T(n-2) + c_2, & \text{se } n > 2 \end{cases}$$

Complexidade:

$$\Omega(2^{n/2}) \leq T(n) \leq O(2^n)$$

FUNÇÃO DE CUSTO E COMPLEXIDADE

Função de custo:

$$T(n) = \begin{cases} c_1, & \text{se } n \leq 2 \\ T(n-1) + T(n-2) + c_2, & \text{se } n > 2 \end{cases}$$

Assumimos que $T(n-2) \leq T(n-1)$. A nossa equação de recorrência passa a ser:

$$T(n) \leq T(n-1) + T(n-1) + c$$

$$T(n) \leq 2T(n-1) + c$$

$$T(n) \leq 2[2T(n-2) + c] + c$$

$$T(n) \leq 2^2T(n-2) + 2c + c$$

$$T(n) \leq 2^2[2T(n-3) + c] + 2c + c$$

$$T(n) \leq 2^3T(n-3) + 2^2c + 2c + c$$

$$T(n) \leq 2^kT(n-k) + c(2^{k-1} + 2^{k-2} + \dots + 2^1 + 2^0)$$

$$T(n) \leq 2^nT(0) + c(2^{n-1} + \dots + 1)$$

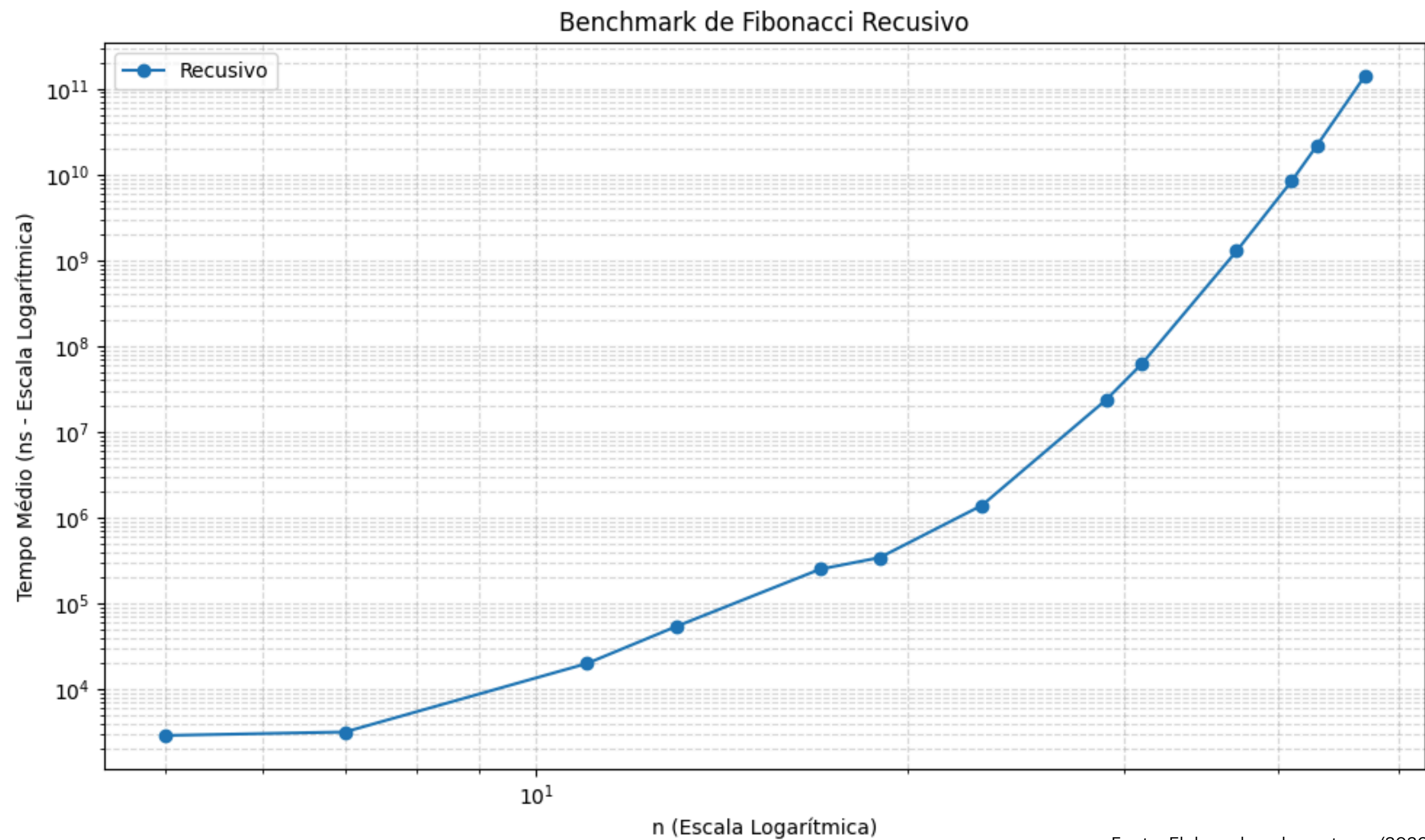
$$T(n) \leq 2^n \cdot O(1) + c(2^n - 1)$$

$$T(n) = O(2^n)$$

EXPERIMENTOS CONDUZIDOS

- **Ambiente de Coleta:** Validação experimental baseada na coleta de tempos de execução em linguagem C.
- **Mitigação de Vieses:** Execução de um número primo de iterações para cada entrada. Isso garante que o tempo reflita o desempenho real, evitando otimizações de cache ou sincronização de hardware.
- **Amostragem Sistemática:** Utilização de 26 pontos de dados distintos para a construção do benchmark.
- **Escala de Teste:** A abrangência vai desde os casos base iniciais até os limites de processamento.
- **Análise Visual:** Geração de gráficos de linha para avaliar o comportamento assintótico e a eficiência de cada algoritmo.

RESULTADOS OBTIDOS



Valores testados: 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47

Fonte: Elaborado pelos autores (2026).

IMPLEMENTAÇÃO MAIS EFICIENTE: FAST DOUBLING

Ideialização: Matriz geradora de Fibonacci.

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^k = \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix}$$

Para dobrar o índice para $2k$, multiplicamos a matriz por ela mesma ($A_{2k}=A_k \cdot A_k$):

$$\begin{pmatrix} F_{2k+1} & F_{2k} \\ F_{2k} & F_{2k-1} \end{pmatrix} = \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix} \cdot \begin{pmatrix} F_{k+1} & F_k \\ F_k & F_{k-1} \end{pmatrix}$$

Multiplicando as linhas pelas colunas e substituindo $F_{k-1}=F_{k+1}-F_k$

Cálculo Par: $F_{2k} = F_k \cdot (2F_{k+1} - F_k)$

Cálculo Ímpar: $F_{2k+1} = F_{k+1}^2 + F_k^2$

Fonte: Fórmulas originais por Édouard Lucas (1878) baseadas em identidades lineares.
Complexidade algorítmica documentada na GNU Multiple Precision Arithmetic Library (GMP).

IMPLEMENTAÇÃO MAIS EFICIENTE: FAST DOUBLING

```
função fast_doubling(n):  
    se n == 0:  
        retorne (0, 1)  
  
    (a, b) = fast_doubling(n / 2)    // a = F(k), b = F(k+1)  
  
    c = a * (2 * b - a)              // Calcula F(2k)  
    d = a * a + b * b                // Calcula F(2k+1)  
  
    se n for par:  
        retorne (c, d)  
    senão:  
        retorne (d, c + d)
```

Fonte: Elaborado pelos autores (2026).

FUNÇÃO DE CUSTO E COMPLEXIDADE

Função de custo:

$$T(n) = T(n/2) + M(n)$$

Complexidade:

$$\Theta(1 \log n)$$

FUNÇÃO DE CUSTO E COMPLEXIDADE

Função de custo:

$$T(n) = T(n/2) + M(n)$$

$$a = 1, b = 2 \text{ e } k = 0$$

$$n^k = 1$$

$$\text{Caso 2: } \log_2 1 = 0$$

Complexidade:

$$\Theta(n^k \log n)$$

$$\Theta(1 \log n)$$

Método mestre

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$\text{caso 1: } \log_b a > k \rightarrow \Theta(n^{\log_b a})$$

$$\text{caso 2: } \log_b a = k \rightarrow \Theta(n^k \log n)$$

$$\text{caso 3: } \log_b a < k \rightarrow \Theta(f(n))$$

NÚMEROS DE PRECISÃO ARBITRÁRIA

- **Aritmética de Precisão Arbitrária (Bignum):** Método necessário para evitar o overflow e permitir o cálculo exato de valores de alta magnitude (BRENT; ZIMMERMANN, 2010).
- **Alocação Dinâmica:** O tamanho máximo do número processado depende apenas da memória RAM disponível no sistema.
- **Superação de Limites:** Rompe com as restrições fixas da arquitetura padrão do processador (como limites de 32 ou 64 bits).

GMP - GNU Multiple Precision Arithmetic Library



Fonte: Imagem da Wikipedia (2026).

FUNÇÃO DE CUSTO E COMPLEXIDADE CONSIDERANDO KARATSUBA

Função de custo:

$$T(n) = T(n/2) + \Theta(n^{\log_2 3})$$

$$a = 1, b = 2 \text{ e } k = \log_2 3$$

$$n^k = n^{\log_2 3}$$

$$\text{Caso 3: } \log_2 1 < \log_2 3$$

Complexidade:

$$\Theta(n^{\log_2 3}).$$

Método mestre

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$\text{caso 1: } \log_b a > k \rightarrow \Theta(n^{\log_b a})$$

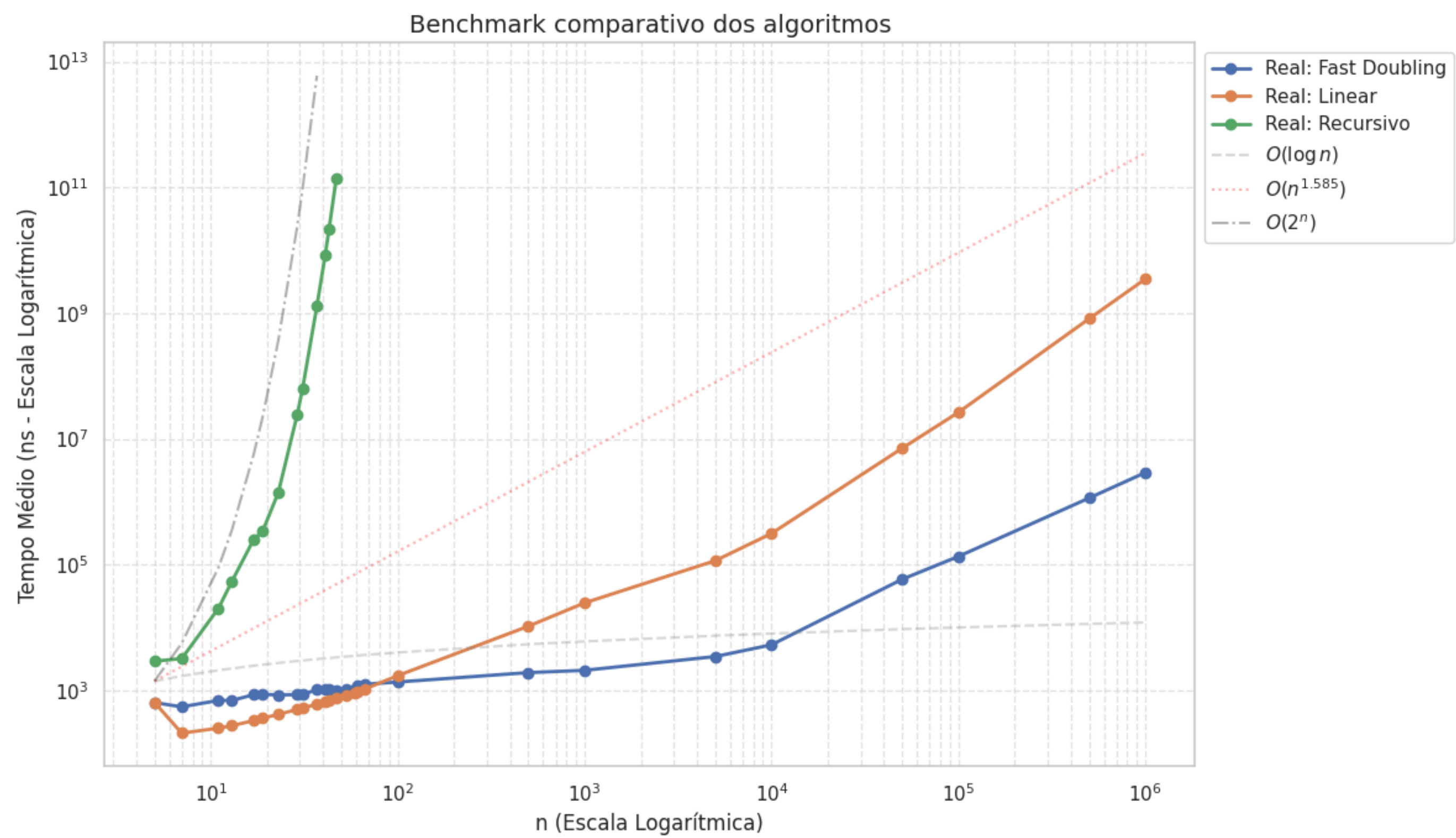
$$\text{caso 2: } \log_b a = k \rightarrow \Theta(n^k \log n)$$

$$\text{caso 3: } \log_b a < k \rightarrow \Theta(f(n))$$

Tendo a complexidade do algoritmo Karatsuba como: $\Theta(n^{\log_2 3})$.

(FEOFILOFF, 2020)

COMPARAÇÃO ENTRE ALGORITMOS



Valores testados: 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, **47**, 53, 59, 61, **67**, 100, 500, 1000, 5000, 10000, 50000, 100000, 500000, 1000000

Fonte: Elaborado pelos autores (2026).

CONCLUSÃO

- **Tipo Primitivo:** O integer overflow ocorre rapidamente (logo após o 46º termo em tipos inteiros comuns).
- **Otimização Matemática:** O método Fast Doubling provou ser superior aos lineares e recursivos.
- **Resultado Final:** Redução da complexidade computacional para $\Omega(M(n))$, permitindo o cálculo de milhões de termos de forma altamente eficiente.

REFERÊNCIAS

BRENT, R. P.; ZIMMERMANN, P. **Modern Computer Arithmetic**. Cambridge: Cambridge University Press, 2010.

LUCAS, Édouard. Théorie des fonctions numériques simplement périodiques. **American Journal of Mathematics**, Baltimore, v. 1, n. 2, p. 184-240, 1878.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier, 2009.

KNUTH, D. E. **The Art of Computer Programming, Volume 1: Fundamental Algorithms**. 3. ed. Reading, Massachusetts: Addison-Wesley, 1997.

SEACORD, R. C. Secure Coding in C and C++. 2. ed. Boston: Addison-Wesley Professional, 2013.

WIKIPEDIA. **GNU Multiple Precision Arithmetic Library**. Disponível em: https://en.wikipedia.org/wiki/GNU_Multiple_Precision_Arithmetic_Library. Acesso em: 23 abr. 2026.

FEOFILOFF, Paulo. **O algoritmo de Karatsuba para multiplicação de números grandes**. Análise de Algoritmos. São Paulo: Instituto de Matemática e Estatística da USP (IME-USP), 23 set. 2020. Disponível em: https://www.ime.usp.br/~pf/analise_de_algoritmos/aulas/karatsuba.html. Acesso em: 30 abr. 2026.